

Syntro

Technical Roadmap & Architecture Blueprint

MVP → Scale · v1.0 · May 2026 · Internal Engineering Document

"The user brings the content. The platform brings the trend."

10	~\$15	2 min	5+
Day MVP Sprint	Est. Month 1 Cost	Generation Target	Beta Users Goal

This document is the single source of truth for the Syntro engineering team. It covers product vision, system architecture, data pipelines, technology stack with cost breakdown, MVP sprint plan (10 days), and the scaling roadmap including SOTA video/AI model integration. Written from a CTO-level perspective for a student-founded startup targeting creators and clothing brands in emerging markets.

Team	Stack	Stage	Market
2 Engineers (Full-Stack)	Next.js · FastAPI · Supabase	Pre-seed / MVP	Nepal → South Asia → Global

Table of Contents

01	Product Vision & Problem Statement
02	Core Architecture Overview
03	Technology Stack & Rationale
04	System Data Flow & Pipelines
05	Database Schema (Supabase / PostgreSQL)
06	Editing & AI Media Pipeline
07	Trend Intelligence Engine
08	Security & Infrastructure
09	MVP Sprint Plan — 10 Days
10	Cost Breakdown (MVP & Scale)
11	Success Metrics & KPIs
12	Scaling Roadmap — Phase 2 & Beyond
13	SOTA Video & AI Model Integration
14	Engineering Rules & CTO Notes

01 — Product Vision & Problem Statement

What We Are Building

Syntro is a trend-aware media editing platform built for clothing brands, solo creators, and social-first businesses. The platform detects what is visually trending on Instagram and TikTok right now, then automatically edits the user's own photos and videos to match those trends — color grading, filters, captions, aspect ratios, and transitions. No generative AI creating fake images. Real media. Real brands. Trend-optimized automatically.

The Core Problem

Pain Point	Reality	Impact
Slow trend response	Trends peak in 24–72 hours; editing takes days	Missed virality windows
High editing cost	Hiring editors is \$500–2000/month for SMBs	Unaffordable for small brands
AI feels fake	Generated images lack real products and people	Low engagement & trust
No integrated workflow	Trend spotting and editing are completely separate	Friction kills momentum
Inconsistent brand	Quick edits break brand identity	Diluted brand equity

Our Solution — The 5-Step Loop

Every user interaction follows a single, repeatable loop designed to be completed in under 2 minutes:

1	Upload	User uploads raw photo or video (JPG, PNG, MP4, MOV). Max 500 MB. Stored in Supabase Storage / Cloudflare R2.
2	Detect	Platform shows trending visual styles scraped from Instagram & TikTok — color palettes, caption styles, audio trends, aspect ratios.
3	Edit	AI applies trend-style editing to the user's media: color grading (LUT-based), filters, caption overlay, aspect ratio conversion.
4	Export	One-click export as MP4 or JPG/PNG. Output optimized per platform: 9:16 Reels/TikTok, 1:1 feed, 4:5 portrait.
5	Analyze	After posting, track engagement per trend style. Platform learns what works for each brand over time.

What We Are NOT Building (Scope Guard)

- Generative AI image tool — no creating images from scratch
- Canva clone — no complex drag-and-drop design editor
- Full video production suite — no timeline editors or keyframing
- Social media scheduler — Phase 1 is export only
- Marketplace or collaboration system — Phase 4+

Target Market

Segment	Profile	Primary Need	Geography
Clothing brands	Small–medium, IG/TikTok focused	Product showcases, lookbooks	Nepal → India → SE Asia
Solo creators	Fashion, lifestyle, streetwear	Daily trend-matching content	Tier 1–2 cities
Agencies	Managing 5–20 brand accounts	High-volume editing at speed	Urban centers
Social commerce	Sell via Instagram / TikTok Shop	Shoppable content, quick edits	Emerging markets

02 — Core Architecture Overview

Architectural Philosophy

We use a modular monolith for the MVP. This is the correct choice for a 2-person student team: single codebase, easy debugging, low infrastructure cost, and zero distributed systems overhead. The monolith is organized into clearly separated modules (auth, trends, editing, analytics, exports) so that microservice extraction is clean when traffic demands it. We never pay for complexity we haven't earned.

System Architecture Diagram

Layer	Component	Technology	Responsibility
Frontend	Dashboard / UI	Next.js 14 + TailwindCSS + shadcn/ui	Upload, trend browse, preview, export, analytics
API Gateway	REST API	FastAPI + Pydantic (Python 3.11)	Auth, routing, validation, business logic
Job Queue	Async Workers	Celery + Redis (Upstash free tier)	Editing jobs, trend scraping, background tasks
AI Editing	Media Processor	FFmpeg + Pillow (+ Replicate optional)	Color grade, captions, resize, format convert
Trend Engine	Scraper + Scorer	Custom scrapers + heuristic scoring	Detect, rank, categorize trending styles
Storage	Object Store	Supabase Storage (S3-compatible)	Raw uploads + edited outputs + CDN delivery
Database	Persistent State	PostgreSQL via Supabase	Users, brand kits, trends, jobs, analytics
Auth	Identity	Supabase Auth (JWT)	Registration, login, session, protected routes
Monitoring	Observability	UptimeRobot + Flower	Uptime alerts, queue dashboard
Deployment	Hosting	Vercel (FE) + Railway (BE + Workers)	Auto-deploy from GitHub, managed Redis

Request Flow — High Level

Every user action follows a predictable path through the system. Understanding this flow is essential for debugging and performance optimization:

Step	Component	Action
1	Browser	User action in Next.js dashboard (upload, edit, export)
2	HTTPS + JWT	Authenticated request sent to FastAPI backend on Railway
3	FastAPI	Route handler validates request, calls appropriate service module
4	Redis Queue	Heavy jobs (editing, trend scraping) are enqueued immediately — API returns job_id
5	Celery Worker	Worker picks up job, downloads media, runs FFmpeg/Pillow pipeline
6	Supabase Storage	Edited file uploaded to Supabase Storage, CDN URL returned
7	PostgreSQL	CDN URL and job metadata saved to database
8	Frontend Poll	Next.js polls /jobs/{id}/status every 3s, displays result on completion

Backend Module Structure

The FastAPI backend is organized as a modular monolith. Each module owns its routes, services, and models. Shared code lives in /core. This structure allows future extraction into microservices without a rewrite:

```
backend/ ■■■ app/ ■ ■■■ api/ ■ ■ ■■■ v1/ ■ ■ ■■■ auth.py # Login, register, refresh ■ ■ ■■■
brand_kit.py # CRUD for brand identities ■ ■ ■■■ trends.py # Trend feed, categories ■ ■ ■■■
editing.py # Submit edit jobs, status ■ ■ ■■■ exports.py # Signed download URLs ■ ■ ■■■ analytics.py
# Engagement tracking ■ ■■■ services/ ■ ■ ■■■ editing_service.py # FFmpeg + Pillow orchestration ■ ■
■■■ trend_service.py # Trend scoring + categorization ■ ■ ■■■ storage_service.py # Supabase Storage
upload/download ■ ■ ■■■ ai_service.py # Replicate/Veo API calls (Phase 2) ■ ■■■ workers/ ■ ■ ■■■
edit_worker.py # Celery task: process media ■ ■ ■■■ trend_worker.py # Celery task: scrape trends ■ ■■■
models/ # SQLAlchemy ORM models ■ ■■■ schemas/ # Pydantic request/response schemas ■ ■■■ core/ ■ ■
■■■■ config.py # Settings from .env ■ ■ ■■■ security.py # JWT verification ■ ■ ■■■ database.py # DB
session factory ■ ■■■ main.py # FastAPI app factory ■■■ alembic/ # DB migrations ■■■ tests/ ■■■
Dockerfile
```

03 — Technology Stack & Rationale

Every technology choice is optimized for two constraints: (1) a 2-person student team with limited time, and (2) a near-zero budget for MVP. We default to managed services, free tiers, and battle-tested libraries. We only self-host when there is no other option.

Layer	Technology	Why We Chose It	Free Tier / Cost
Frontend	Next.js 14 + TypeScript	App Router, SSR, great DX, huge ecosystem	Free on Vercel
UI Components	TailwindCSS + shadcn/ui	Fast beautiful UI without custom CSS	Free / OSS
Backend	FastAPI (Python 3.11)	Async, fast, Pydantic validation, ML-friendly	Free (Railway ~\$5/mo)
ORM	SQLAlchemy 2.0 + Alembic	Type-safe queries, clean migrations	Free / OSS
Database	Supabase (PostgreSQL)	Managed DB + Auth + Storage in one, generous free tier	Free up to 500 MB
Auth	Supabase Auth	Built into Supabase, handles JWT, sessions, OAuth	Free up to 50K MAU
Storage	Supabase Storage	S3-compatible, CDN delivery, same dashboard as DB	1 GB free
Job Queue	Celery + Redis (Upstash)	Battle-tested async processing, free Redis tier	\$0 (10K cmds/day)
Image Editing	Pillow (PIL)	Filters, color grading, overlays, captions	Free / OSS
Video Editing	FFmpeg	Industry standard, LUTs, captions, resize, format	Free / OSS
AI (optional MVP)	Replicate API	Pay-per-use, no GPU setup, Flux/SDXL access	~\$0.003–0.05/call
Video AI (Phase 2)	Google Veo 3 API	SOTA video gen, cheaper than self-hosting	\$0.35–0.70/video
Frontend Deploy	Vercel	Auto-deploy from GitHub, edge CDN, free tier	Free hobby plan
Backend Deploy	Railway	Simple Python deploy, env vars, managed services	\$5/mo starter
Monitoring	UptimeRobot + Flower	Uptime alerts + Celery queue dashboard	Free tiers
CI/CD	GitHub Actions	Auto-test on push, free for public repos	Free 2K min/mo

Why Supabase Over Separate Services

For a student team, Supabase is a force multiplier. It replaces four separate services (PostgreSQL hosting, Auth service, Object Storage, and Realtime subscriptions) with a single dashboard and SDK. The free tier is generous enough to cover the entire MVP phase. Migration to dedicated services (Neon DB, Clerk, Cloudflare R2) is straightforward if needed at scale.

Why FFmpeg + Pillow Before AI Models

The editing quality needed for MVP is achievable with deterministic processing. FFmpeg LUT color grading + Pillow filter overlays produce professional-looking results at exactly \$0 compute cost. AI style transfer via Replicate is a Phase 2 upgrade, not a starting point. This approach keeps the job completion time under 30 seconds and cost near zero per edit.

04 — System Data Flow & Pipelines

4.1 — Media Upload Pipeline

Media upload is designed to never proxy large files through the FastAPI backend. The backend only generates a signed URL; the browser uploads directly to Supabase Storage. This keeps the backend stateless and prevents memory overflow on large video files.

Step	Actor	Action	Technology
1	Frontend	User selects file, client requests upload URL	Next.js fetch()
2	FastAPI	Validate file type/size, generate signed upload URL	Supabase Storage SDK
3	Browser	Upload file directly to Supabase Storage	Direct PUT to signed URL
4	FastAPI	Receive upload confirmation, create media_asset record in DB	SQLAlchemy + Supabase
5	FastAPI	Enqueue preprocessing job, return asset_id to frontend	Celery task
6	Worker	Download file, extract metadata (resolution, fps, duration, mime)	FFprobe + Pillow
7	Worker	Update media_asset record with metadata, set status=ready	SQLAlchemy
8	Frontend	Display uploaded file preview with metadata	Next.js state update

4.2 — Editing Job Pipeline

The editing pipeline is the core of the product. Every step is independently retryable via Celery. If a step fails, the job returns to the queue with exponential backoff — no manual intervention needed.

Step	Stage	Tool	Description
1	Enqueue	FastAPI + Celery	POST /edit receives brand_kit_id + trend_id + asset_id, returns job_id immediately
2	Download	Supabase Storage SDK	Worker fetches raw media from storage to temp disk
3	Preprocess	FFprobe + Pillow	Extract resolution, duration, FPS, color profile, orientation
4	Trend Map	Trend Service	Load trend parameters: LUT file, filter settings, caption style, aspect ratio target
5	Color Grade	FFmpeg LUTs	Apply 3D LUT matching trend's color palette (warm/cool/moody/Y2K/minimal)
6	Filter Overlay	Pillow	Apply grain, vignette, light leak effects; adjust brightness/contrast/saturation
7	Caption Burn	FFmpeg drawtext	Render trend-style caption with brand font and colors onto image or video
8	Aspect Ratio	FFmpeg	Auto-crop/pad to target: 9:16 (Reels/TikTok), 1:1 (Feed), 4:5 (Portrait)
9	Render	FFmpeg	Final encode: MP4 (H.264, AAC) for video; JPEG/WebP for images
10	Optimize	FFmpeg	Compress to platform-optimal bitrate; strip metadata for privacy
11	Upload	Supabase Storage	Upload edited file to /edited/ path, get CDN URL
12	Complete	FastAPI + Redis	Update edit_job status=complete, store CDN URL; frontend notified

4.3 — Trend Detection Dataflow

The trend pipeline runs as a background Celery beat job every 6 hours. In MVP, we seed the database with 5 hardcoded trend categories and supplement with lightweight scraping. The full automated pipeline deploys in Phase 2.

Step	Process	Source / Tool	Output
1	Ingest	Instagram/TikTok API or lightweight scraper	Raw hashtags, audio names, caption samples
2	Score	Heuristic formula (see Section 07)	Trend score 0.0–1.0 per style
3	Categorize	Rule-based tagging	Aesthetic labels: moody, Y2K, minimal, bright, cottagecore
4	Deduplicate	PostgreSQL UPSERT	Remove duplicate trends across scraping runs
5	Store	PostgreSQL trends table	Scored, labeled trend records with TTL (expires_at)
6	Serve	FastAPI /trends endpoint	Ranked trend cards displayed in the dashboard
7	Map to LUT	Trend Service	Link trend aesthetic to FFmpeg LUT file + Pillow settings

05 — Database Schema (Supabase / PostgreSQL)

All tables live in a single PostgreSQL database on Supabase. Supabase Row Level Security (RLS) is enabled on all user-owned tables. The schema is designed for MVP simplicity with clear extension points for Phase 2 features.

Table: users

Column	Type / Constraint	Description
id	UUID PRIMARY KEY	Supabase Auth user ID
email	TEXT UNIQUE NOT NULL	User email address
name	TEXT	Display name
subscription_tier	TEXT DEFAULT 'free'	free starter pro
created_at	TIMESTAMPTZ DEFAULT NOW()	Registration timestamp
updated_at	TIMESTAMPTZ	Last profile update

Table: brand_kits

Column	Type / Constraint	Description
id	UUID PRIMARY KEY DEFAULT gen_random_uuid()	Unique brand kit ID
user_id	UUID REFERENCES users(id)	Owner user
name	TEXT NOT NULL	Brand name
logo_url	TEXT	CDN URL of logo asset
primary_color	TEXT	Hex color #RRGGBB
secondary_color	TEXT	Hex color #RRGGBB
font_name	TEXT	Font family name
tone	TEXT	Brand tone: bold minimal playful luxury
created_at	TIMESTAMPTZ DEFAULT NOW()	Creation timestamp

Table: media_assets

Column	Type / Constraint	Description
id	UUID PRIMARY KEY DEFAULT gen_random_uuid()	Asset ID
user_id	UUID REFERENCES users(id)	Uploader
storage_key	TEXT NOT NULL	Supabase Storage path
cdn_url	TEXT	Public CDN URL
mime_type	TEXT	video/mp4 image/jpeg etc.
duration_sec	FLOAT	Video duration in seconds (null for images)

Column	Type / Constraint	Description
resolution_w	INT	Width in pixels
resolution_h	INT	Height in pixels
fps	FLOAT	Frames per second (video only)
file_size_bytes	BIGINT	File size
status	TEXT DEFAULT 'pending'	pending ready error
created_at	TIMESTAMPTZ DEFAULT NOW()	Upload timestamp

Table: trends

Column	Type / Constraint	Description
id	UUID PRIMARY KEY DEFAULT gen_random_uuid()	Trend ID
name	TEXT NOT NULL	Trend name e.g. Soft Moody Film
category	TEXT	Aesthetic category label
platform	TEXT	instagram tiktok both
aesthetic_tags	TEXT[]	Array of style tags
score	FLOAT	Heuristic score 0.0–1.0
lut_file	TEXT	Filename of FFmpeg LUT to apply
pillow_settings	JSONB	JSON: brightness, contrast, saturation, grain
caption_style	JSONB	JSON: font_size, position, color, opacity
detected_at	TIMESTAMPTZ	When trend was identified
expires_at	TIMESTAMPTZ	Auto-expire stale trends

Table: edit_jobs

Column	Type / Constraint	Description
id	UUID PRIMARY KEY DEFAULT gen_random_uuid()	Job ID
user_id	UUID REFERENCES users(id)	Requester
media_asset_id	UUID REFERENCES media_assets(id)	Input media
brand_kit_id	UUID REFERENCES brand_kits(id)	Brand to apply
trend_id	UUID REFERENCES trends(id)	Trend style to apply
status	TEXT DEFAULT 'queued'	queued processing complete failed
edited_url	TEXT	CDN URL of output file
aspect_ratio	TEXT	9:16 1:1 4:5
error_message	TEXT	Error detail if failed

Column	Type / Constraint	Description
celery_task_id	TEXT	Celery task UUID for polling
started_at	TIMESTAMPTZ	Worker pick-up time
completed_at	TIMESTAMPTZ	Completion time
created_at	TIMESTAMPTZ DEFAULT NOW()	Submission time

Table: exports

Column	Type / Constraint	Description
id	UUID PRIMARY KEY DEFAULT gen_random_uuid()	Export ID
edit_job_id	UUID REFERENCES edit_jobs(id)	Source edit job
format	TEXT	mp4 jpg png webp
resolution	TEXT	e.g. 1080x1920
aspect_ratio	TEXT	9:16 1:1 4:5
file_url	TEXT	Signed download URL
downloaded_at	TIMESTAMPTZ	When user downloaded
created_at	TIMESTAMPTZ DEFAULT NOW()	Export creation time

Table: analytics

Column	Type / Constraint	Description
id	UUID PRIMARY KEY DEFAULT gen_random_uuid()	Record ID
user_id	UUID REFERENCES users(id)	User
edit_job_id	UUID REFERENCES edit_jobs(id)	Source edit
trend_id	UUID REFERENCES trends(id)	Trend used
platform_posted	TEXT	instagram tiktok youtube
likes	INT DEFAULT 0	Engagement: likes
views	INT DEFAULT 0	Engagement: views
shares	INT DEFAULT 0	Engagement: shares
comments	INT DEFAULT 0	Engagement: comments
recorded_at	TIMESTAMPTZ DEFAULT NOW()	When data was logged

06 — Editing & AI Media Pipeline

The editing pipeline is the product's core value delivery mechanism. Phase 1 uses entirely deterministic processing — no AI API calls required. This keeps cost at \$0 per edit and latency under 30 seconds for images, under 90 seconds for videos under 60 seconds.

Phase 1 Editing Capabilities (Deterministic — \$0 Cost)

Edit Type	Tool	Description	Est. Time
Color Grading	FFmpeg LUTs	Apply 3D Look-Up Tables matching trend palette: warm film, moody blue, Y2K soft, soft vintage	2-5 sec
Filter Overlay	Pillow	Grain texture, vignette edges, light leak, halation effects popular in current aesthetics	1-3 sec
Caption Burn	FFmpeg drawtext	Overlay trend-style text with brand font, position, color, and opacity onto image or video	1-2 sec
Aspect Ratio	FFmpeg scale+pad	Smart crop or letterbox to 9:16 (Reels/TikTok), 1:1 (Feed), 4:5 (Portrait feed)	1-8 sec
Brightness/Contrast	Pillow	Match luminance profile of trending aesthetic; apply curves adjustment	1 sec
Saturation/Tone	Pillow ImageEnhance	Desaturate for moody look; boost for vibrant Y2K aesthetic	1 sec
Video Fade	FFmpeg	Fade-in / fade-out transitions for video clips	2-5 sec
Optimize & Compress	FFmpeg	H.264 MP4 at platform-optimal bitrate (8-12 Mbps for 1080p)	5-15 sec

Key FFmpeg Commands (Reference)

Apply LUT + resize to 9:16:

```
ffmpeg -i input.mp4 -vf "lut3d=warm_film.cube,scale=1080:1920:force_original_aspect_ratio=decrease,pad=1080:1920:-1:-1" -c:v libx264 -crf 18 out.mp4
```

Burn caption text:

```
ffmpeg -i input.mp4 -vf "drawtext=text='Trending Now':fontfile=brand.ttf:fontsize=48:fontcolor=white:x=(w-text_w)/2:y=h*0.85" out.mp4
```

Fade in/out:

```
ffmpeg -i input.mp4 -vf "fade=t=in:st=0:d=0.5,fade=t=out:st=4.5:d=0.5" out.mp4
```

Convert to 1:1 square:

```
ffmpeg -i input.mp4 -vf "crop=min(iw\,ih):min(iw\,ih),scale=1080:1080" out.mp4
```

Phase 2 AI Upgrades (API-Based, Pay-Per-Use)

These capabilities are NOT in the MVP. They are integrated in Phase 2 once FFmpeg/Pillow edits are proven insufficient for user retention. Always API-based — never self-hosted GPU.

Capability	API	Cost Estimate	Phase
AI Style Transfer	Replicate (SDXL Style)	~\$0.05–0.15 per image	Phase 2
Background Removal	Replicate (RMBG-2.0)	~\$0.005 per image	Phase 2
Auto Subtitles	OpenAI Whisper API	~\$0.006 per minute audio	Phase 2
AI Video Generation	Google Veo 3 API	~\$0.35–0.70 per 5-sec clip	Phase 3
Video Enhancement	Topaz Video AI API	~\$0.02–0.10 per minute	Phase 3
Face Enhancement	CodeFormer via Replicate	~\$0.01–0.03 per image	Phase 3

07 — Trend Intelligence Engine

The trend intelligence system is the product's competitive moat. Editing quality is commodity — FFmpeg filters are free and open source. What competitors cannot easily replicate is a proprietary trend detection and scoring system trained on thousands of data points from real user engagement. We start simple and compound.

Trend Score Formula

$$\text{trend_score} = (\text{engagement_velocity} \times 0.35) + (\text{hashtag_growth} \times 0.25) + (\text{audio_reuse} \times 0.20) + (\text{caption_similarity} \times 0.20)$$

Signal	Weight	Source	How Measured
Engagement Velocity	35%	Platform metrics	Likes + shares per hour / follower count
Hashtag Growth	25%	Hashtag tracking	% increase in hashtag uses over 24h window
Audio Reuse	20%	Audio fingerprint	Number of videos using same audio track in 48h
Caption Similarity	20%	NLP / pattern match	Semantic similarity of captions to template patterns

MVP Trend Categories (Hardcoded — Phase 1)

For the MVP, we hardcode 5–8 trend categories manually curated by the team. Each category maps directly to an FFmpeg LUT file and Pillow settings JSON. This gives immediate value without requiring a scraping infrastructure:

Category	Aesthetic	LUT File	Pillow Settings	Typical Platform
Soft Moody Film	Low saturation, warm shadows, grain	soft_film.cube	sat:0.7, grain:0.4, vignette:0.3	Instagram
Y2K Vivid	High saturation, cyan/magenta boost	y2k_vivid.cube	sat:1.6, brightness:1.1, sharpen:1.2	TikTok
Minimal Clean	Bright, high contrast, desaturated	minimal_clean.cube	sat:0.5, contrast:1.3, brightness:1.15	Both
Golden Hour	Warm orange tones, soft highlights	golden_hour.cube	sat:1.1, warmth:+30, vignette:0.2	Instagram
Dark Academia	Cool shadows, muted colors, vintage	dark_academia.cube	sat:0.6, coolness:+20, grain:0.5	TikTok

Phase 2 — Automated Trend Ingestion

In Phase 2, a Celery beat scheduler runs trend ingestion every 6 hours. We use a combination of lightweight scrapers (Instagram Explore, TikTok trending page) and official APIs where available. The scoring formula is applied automatically and results stored with a 72-hour TTL.

Phase 3 — Embedding-Based Intelligence

In Phase 3, we introduce pgvector in Supabase to store aesthetic embeddings. Every trend is encoded as a vector using CLIP or a fine-tuned visual encoder. This enables: (1) semantic trend similarity search, (2) personalized trend recommendations per brand, (3) regional trend segmentation (Nepal trends differ from India trends), and (4) virality prediction models trained on our accumulated engagement data. This is the point where the data moat becomes unassailable.

08 — Security & Infrastructure

Security Layer	Implementation	Why It Matters
Authentication	Supabase Auth JWT tokens on all API routes	Prevents unauthorized API access
Row Level Security	Supabase RLS policies: users see only their data	Database-level data isolation
Signed Upload URLs	Signed URLs for direct-to-storage uploads	Backend never handles large file bytes
FFmpeg Sandboxing	Workers run in Docker containers with limited syscalls	Prevents malicious media exploits
Upload Validation	MIME type check + magic byte verification + size limit	Prevents malware disguised as media
Rate Limiting	10 edit jobs/min per user via FastAPI middleware	Prevents API abuse and cost explosion
Worker Isolation	Celery workers run in separate containers	Job failures don't crash the API
Temp File Cleanup	Workers delete temp files after job completion	Prevents disk exhaustion and data leaks
HTTPS Only	Vercel + Railway enforce TLS, no HTTP fallback	Encrypts all data in transit
Environment Secrets	All API keys in Railway/Vercel env vars, never in code	No credential leakage in GitHub
CORS	FastAPI CORS middleware restricted to frontend origin	Prevents cross-origin attacks

Critical Rule: Never Proxy Media Through the Backend

Large media files (videos up to 500 MB) must NEVER pass through the FastAPI process. The backend generates signed URLs and clients upload/download directly to Supabase Storage. Violating this rule causes memory exhaustion, slow response times, and Railway cost spikes. This is the single most common mistake in media platform architecture.

09 — MVP Sprint Plan (10 Days)

Two engineers. Ten days. One working product with real users. This plan is aggressive but achievable if scope is ruthlessly defended. Every day has a specific deliverable. If a task is not done by end of day, it either gets cut or moved — never expanded.

Day	YOU — Backend / Infra	FRIEND — Frontend / UI	✓ Deliverable
1	GitHub monorepo setup, .env structure, Docker Compose for local dev.	React + Tailwind CSS boilerplate, TypeScript, Firebase, and Supabase integration.	Repo, Docker, and local dev environment ready.
2	Provision Supabase project. Write all SQLAlchemy models and database schema.	Initial database schema, Auth, and User models.	Supabase project, SQLAlchemy models, and database schema.
3	FastAPI routes: /auth, /brand-kit CRUD, /trend API (placeholder).	Auth, User, and BrandKit components.	Auth, User, and BrandKit endpoints and components.
4	Mock trend data endpoint. Seed DB with 5 trending items.	Initial category navigation and trending items display.	Mock trend data endpoint and seeded database.
5	File upload endpoint: validate MIME + size. Generate signed Supabase Storage URLs.	Image upload form and image gallery display.	File upload endpoint and signed Supabase Storage URLs.
6	Image editing pipeline: FFmpeg LUT color grade + video player.	Edit preview filters and video player.	Image editing pipeline and video player.
7	Video editing pipeline: FFmpeg color grade + video player.	Video editing pipeline and video player.	Video editing pipeline and video player.
8	Celery + Upstash Redis integration. Convert jobs to async tasks.	Initial job queue and progress indicator.	Celery + Upstash Redis integration and job queue.
9	Export endpoint: generate signed download URL from Supabase Storage.	Initial export button and download progress indicator.	Export endpoint and signed download URL.
10	End-to-end integration bugs. Input validation and error handling.	Initial end-to-end integration and error handling.	End-to-end integration and error handling.
11-12	Deploy backend to Railway: configure env vars, Supabase URL, Redis connection.	Deploy frontend to Vercel: configure env vars, Supabase URL, Redis connection.	Backend and frontend deployed to production.
13-14	Analytics endpoint: log edit_jobs count, export count, feedback.	Initial analytics dashboard and feedback form.	Analytics endpoint and initial analytics dashboard.

Sprint Engineering Rules

Rule	Reasoning
■ No Microservices	Monolithic FastAPI only. Split later when you have actual scaling pain, not before.
■ No Self-Hosted AI	Use Replicate or API-based models. Never manage GPU infrastructure in MVP.
■ No Premature Optimization	Ship working code first. Profile and optimize in Phase 2.
■ Speed > Perfection	A working ugly edit beats a perfect non-existent one every time.
■ 2-Minute Hard Limit	Every editing flow must complete in under 2 minutes. Non-negotiable.
■ Flag Blockers Fast	If you're stuck > 2 hours, communicate immediately. Don't suffer in silence.
■ Validate on Day 13	Beta users on Day 13–14 are the only true success metric.

10 — Cost Breakdown

Cost discipline is a survival skill for student-founded startups. Every dollar spent before product-market fit is a dollar that could fund another month of runway. The MVP runs on approximately \$5–20/month — less than a Netflix subscription.

Phase 1 — MVP (Month 1–2)

Service	Plan	Cost/Month	Limit / Notes
Supabase (DB + Auth + Storage)	Free tier	\$0	500 MB DB, 1 GB storage, 50K MAU — plenty for MVP
Railway (FastAPI backend)	Starter	\$5	512 MB RAM, enough for FastAPI + 1 Celery worker
Upstash (Redis)	Free tier	\$0	10,000 commands/day — enough for MVP job queue
Vercel (Next.js frontend)	Hobby	\$0	Free for personal projects, auto-deploy from GitHub
UptimeRobot (monitoring)	Free	\$0	5-minute interval uptime checks
GitHub Actions (CI/CD)	Free	\$0	2,000 minutes/month for free accounts
Domain (Namecheap)	Annual	~\$1.25	\$15/year billed annually
Replicate API (optional AI)	Pay-per-use	\$0–15	Only if FFmpeg edits insufficient; ~\$0.05 per AI call
TOTAL		\$5–20/mo	Less than a coffee a week

Phase 2 — Beta Growth (Month 3–4, ~20–50 users)

Service	Plan	Cost/Month	Trigger to Upgrade
Supabase	Pro (\$25/mo)	\$25	When DB hits 400 MB or auth exceeds 48K MAU
Railway (Backend + Workers)	Pro	\$20–40	When CPU > 80% consistently for 48h+
Upstash (Redis)	Pay-as-you-go	\$2–5	When queue exceeds 10K commands/day
Vercel (Frontend)	Hobby → Pro (\$20)	\$0–20	When bandwidth exceeds 100 GB/month
Replicate API (AI edits)	Higher volume	\$20–60	When AI style transfer becomes default edit mode
TOTAL		\$67–150/mo	Upgrade only when metrics justify it

Phase 3 — Scale (Month 5–6, SOTA AI Models, 100+ users)

Service	Plan	Cost/Month	Notes
Supabase Pro	Managed PostgreSQL	\$25	pgvector enabled for embeddings
Railway Pro (2+ workers)	Multiple services	\$40–80	Horizontal Celery worker scaling
Cloudflare R2 (overflow storage)	Usage-based	\$5–15	If Supabase Storage limits hit
Google Veo 3 API	Per video generated	\$50–200	\$0.35–0.70 per 5-sec clip; only on-demand
OpenAI Whisper API	Per minute audio	\$10–30	Auto-subtitles for video content

Service	Plan	Cost/Month	Notes
Replicate (style transfer)	Volume	\$30–100	Background removal + AI style edits
Clerk (Auth upgrade)	Pro if needed	\$25	Only if Supabase Auth limits hit
CDN / Bandwidth	Cloudflare	\$0–10	Cloudflare proxy is free, R2 egress is free
TOTAL		\$185–485/mo	Covered by ~20–30 paying users at \$15–20/mo

Golden Rule: Never Upgrade Before the Metric

Do not spend money on infrastructure before 50+ active users. Do not upgrade Supabase until DB is over 400 MB. Do not add Railway workers until CPU stays above 80% for 48+ hours. Do not use expensive AI APIs until deterministic FFmpeg edits are proven insufficient. Every premature infrastructure spend is runway burned before product-market fit.

11 — Success Metrics & KPIs

<2 min	5+	<10%	>98%
Max Edit Time	Beta Users Day 14	Job Failure Rate	Uptime Target

Metric	Phase 1 Target	Phase 2 Target	Phase 3 Target	How to Measure
Edit completion time	< 2 minutes	< 90 seconds	< 45 seconds	Celery task duration logging
Active beta users	5 brands/creators	20–30 users	100+ users	Daily active users in analytics table
Daily edits generated	Team internal use	50+ edits/day	500+ edits/day	COUNT(edit_jobs) per day
Job failure rate	< 10%	< 3%	< 1%	COUNT(status=failed)/COUNT(total)
Deployment uptime	> 98%	> 99.5%	> 99.9%	UptimeRobot alerts
User retention (W2)	N/A	> 60% return	> 75% return	Users active in week 2 / week 1
Willingness to pay	Qualitative signal	1–3 paying users	20+ paying users	Stripe conversion rate
Cost per edit	< \$0.10	< \$0.05	< \$0.02	Monthly infra cost / total edits
Engagement lift	N/A (track manually)	2–3x vs pre-tool	3–5x vs baseline	User-reported analytics input

12 — Scaling Roadmap — Phase 2 & Beyond

Phase 1 — MVP (Days 1–14)	Ship the core loop
• Upload → Detect → Edit → Export → Analyze functional • Deterministic editing: FFmpeg + Pillow (zero AI cost) • 5 beta users editing daily by Day 14	• 5 hardcoded trend categories with LUT mappings • Supabase DB + Auth + Storage fully integrated • Deployed to Vercel + Railway with CI/CD
Phase 2 — Automation (Weeks 3–6)	Automate trends + improve quality
• Automated trend ingestion via Celery beat (every 6h) • Improved prompt engineering for AI-assisted edits • 20–30 active beta users; early revenue signal (\$10–15/mo) • Publishing workflow: export + platform guide	• Hashtag and audio trend scraping pipeline • Analytics dashboard: engagement per trend style • Queue optimization: target < 90s edit time
Phase 3 — Intelligence (Month 2–3)	Build the defensible moat
• pgvector in Supabase: trend aesthetic embeddings • Personalized trend recommendations per brand • Virality prediction model (trained on our engagement data) • Background removal + auto-subtitles via API	• CLIP-based visual similarity search for trends • Regional trend segmentation: Nepal vs India vs SE Asia • AI style transfer via Replicate (optional upgrade path)
Phase 4 — Scale & Monetize (Month 4–6)	Build the business
• Stripe subscription billing: Free / Starter \$10 / Pro \$20 • Multi-brand support and team collaboration (Pro tier) • Observability stack: Sentry + Datadog or Grafana • Creator marketplace foundations (Phase 5 preview)	• Rate limiting and quota enforcement per tier • Horizontal Celery worker scaling on Railway • 100+ active creators, first recurring revenue
Phase 5 — Platform (Month 6+)	Become infrastructure
• Creator marketplace: sell preset packs and brand kits • White-label solution for social media agencies • Acquisition potential from trend analytics companies	• B2B API: agencies integrate Syntro into their workflow • Regional expansion: dedicated trend models per country • \$500K+ ARR target; Series A readiness

13 — SOTA Video & AI Model Integration

This section covers our strategy for integrating state-of-the-art generative AI models as the product scales. The core principle: use APIs, never self-host. A student team cannot manage GPU infrastructure. Pay-per-use APIs give access to the best models without DevOps overhead. We integrate SOTA only when deterministic editing is proven insufficient.

Why API-First for AI Models

Running SOTA video generation models (Veo 3, Wan 2.1, Kling 3.0) requires H100 GPUs at \$2–8/hour per GPU. For a 5-second video clip, you might need 30–120 seconds of GPU time. Self-hosting at any reasonable scale costs \$3,000–15,000/month before break-even. Google Veo 3 API charges ~\$0.35–0.70 per 5-second clip. At 100 clips/day, that's \$35–70/day — still affordable. The API model wins at every scale until you exceed 10,000 clips/day, at which point you negotiate enterprise GPU contracts.

SOTA Model Evaluation Matrix

Model	Provider	API Available	Cost/Output	Best For	Integration Phase
Veo 3	Google DeepMind	Yes (Vertex AI)	\$0.35–0.70/5s clip	Cinematic video generation	Phase 3
Wan 2.1	Alibaba Cloud	Yes (via Replicate)	\$0.05–0.15/clip	Cost-effective video gen	Phase 3
Kling 3.0	Kuaishou	Yes (kling.ai API)	\$0.10–0.30/clip	High quality video gen	Phase 3
Flux Schnell	Black Forest Labs	Yes (Replicate)	\$6.00/3/image	Fast image generation	Phase 2
SDXL	Stability AI	Yes (Replicate)	\$0.005/image	Style transfer, quality images	Phase 2
Whisper v3	OpenAI	Yes (OpenAI API)	\$0.006/min audio	Auto-subtitles for video	Phase 2
RMBG-2.0	BRIA AI	Yes (Replicate)	\$0.005/image	Background removal	Phase 2
Claude 3.5 Sonnet	Anthropic	Yes	\$3/M input tokens	Caption generation, trend copy	Phase 2
Gemini Flash 2.0	Google	Yes	\$0.075/M tokens	Fast caption + trend description	Phase 2

SOTA Video Pipeline Architecture (Phase 3)

When we integrate video generation, the pipeline extends the existing editing pipeline with an optional generation step. The user can choose: (A) edit their existing video, or (B) generate a trend-matching video clip from a text prompt + brand kit, then apply editing. Option B is always more expensive and slower — it's a premium upsell.

Step	Option A (Edit)	Option B (Generate + Edit)	Tool
1	User uploads existing video	User inputs prompt + selects trend	Next.js UI
2	Preprocess: extract metadata	Build optimized prompt with brand context + trend keywords	FastAPI service
3	Skip generation	Call Veo 3 / Kling API: generate 5-sec clip matching trend + text	Video generation API
4	Apply color grade (LUT)	Apply color grade to generated clip	FFmpeg
5	Apply filter overlay	Apply filter overlay	Pillow

Step	Option A (Edit)	Option B (Generate + Edit)	Tool
6	Burn captions with brand font	Burn trend-style captions	FFmpeg drawtext
7	Resize to target aspect ratio	Resize to 9:16	FFmpeg
8	Optimize + upload to storage	Optimize + upload to storage	FFmpeg + Supabase Storage
9	User previews + exports	User previews + exports	Next.js

AI-Powered Caption Generation (Phase 2)

Instead of users writing captions manually, we use Claude 3.5 Haiku or Gemini Flash to generate trend-optimized captions based on: (1) the selected trend aesthetic, (2) the brand's tone from their brand kit, and (3) the platform (Instagram vs TikTok caption conventions differ). Cost: ~\$0.001–0.005 per caption generation. High value, negligible cost.

System: You generate short social media captions. User: Generate a caption for a clothing brand Instagram post. Trend: Soft Moody Film (low saturation, vintage feel) Brand tone: Minimal, luxury Platform: Instagram Max length: 150 characters Include: 3 relevant hashtags Output: "quiet luxury, redefined.
#softaesthetic #minimalstyle #slowfashion"

14 — Engineering Rules & CTO Notes

■ The Moat Is Trend Intelligence, Not Editing

FFmpeg filters are free. Pillow is free. Anyone can copy your editing pipeline in a weekend. What they cannot copy is 12 months of accumulated engagement data showing exactly which trend aesthetics drive the most views for Nepali clothing brands in September. Build the analytics instrumentation from Day 1. Every engagement data point makes trend detection smarter. This data compound is the only defensible moat.

■ Editing Real Media Beats Generating Fake Media

Brands want their actual products, their actual models, and their actual spaces in content. AI-generated images of generic clothing on generic people perform worse than a real product photo with a trend-optimized filter. This is not a philosophical choice — it is a product decision validated by engagement data. Editing user-owned media is the right call for this market.

■ FFmpeg First. Always.

Do not reach for Replicate AI models until FFmpeg + Pillow edits are proven insufficient by user feedback. Deterministic edits are faster (< 30s vs 60–120s for AI), cheaper (\$0 vs \$0.05–0.70), and more predictable (no hallucinations, no prompt engineering required). AI generation is a Phase 2 upsell, not a Phase 1 requirement.

■ Ship in 10 Days or It Doesn't Matter

The largest engineering mistake at this stage is overbuilding before validating demand. A working MVP that 5 real users touch in 10 days is worth more than a perfect platform that ships in 3 months to zero users. Speed of execution is the only metric that matters right now. 80% quality shipped beats 100% quality delayed every time.

■ Never Upgrade Infrastructure Before the Metric

Railway, Supabase, and Upstash free tiers will handle your first 50 users without breaking a sweat. Don't upgrade to paid tiers before you hit the specific limit. Don't add Celery worker replicas before CPU is consistently above 80%. Premature infrastructure spend burns runway that should fund user acquisition.

■ The Student Budget Advantage

Being a student team is actually an advantage: GitHub Education gives free Pro accounts and partner credits (Railway, Render, MongoDB Atlas). Apply for Google for Startups (\$1,000–10,000 in GCP credits). Apply for AWS Activate (\$5,000 credits). Apply for Anthropic startup credits. These programs can fund your entire Phase 2–3 infrastructure cost at zero cash spend.

■ Analytics Is the Flywheel

Instrument every user action from Day 1: uploads, edits, exports, downloads. Even if users manually input their engagement data (likes, views, shares) for Phase 1, that data becomes the training signal for Phase 3 trend prediction models. The longer you wait to instrument, the longer the flywheel takes to spin up.

■ Communicate Blockers in Under 2 Hours

A 2-person team with a 10-day deadline has zero margin for silent struggling. If either engineer is blocked for more than 2 hours, communicate immediately. Cut scope rather than miss the deadline. A delivered MVP with 80% of features beats a planned MVP with 100% of features that is never shipped.

The platform wins by being the fastest path from

Trend Emergence → Edited Content → Published → Engaged Audience

Build this. Ship in 10 days. Get 5 real users. Listen to them. Iterate.